Lab 9: Data Path, Control Units, and Immediate Generation

Lab Objectives:

- Design Verilog modules for data path, control unit, and immediate generation.
- Implement data path elements.
- Generate control signals for instruction decoding and operation management.
- Develop immediate generation logic.
- Integrate modules for RISC-V processor design.

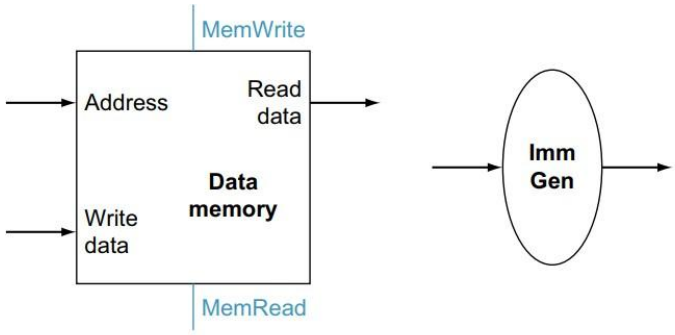
Data Path:

In modern computer architectures, the datapath plays a crucial role in executing instructions efficiently. A datapath comprises various components, including the register file, arithmetic logic unit (ALU), memory unit, and control unit, working together to process instructions. By integrating these components and control signals, you will gain practical insights into processor design and operation.

This unified datapath is designed to execute all instructions within a single clock cycle, ensuring that no datapath resource is utilized more than once per instruction. Hence, any component required multiple times must be duplicated.

To support this unified architecture, we require separate memory units for instructions and data. While some functional units need duplication, many elements can be shared across different instruction flows.

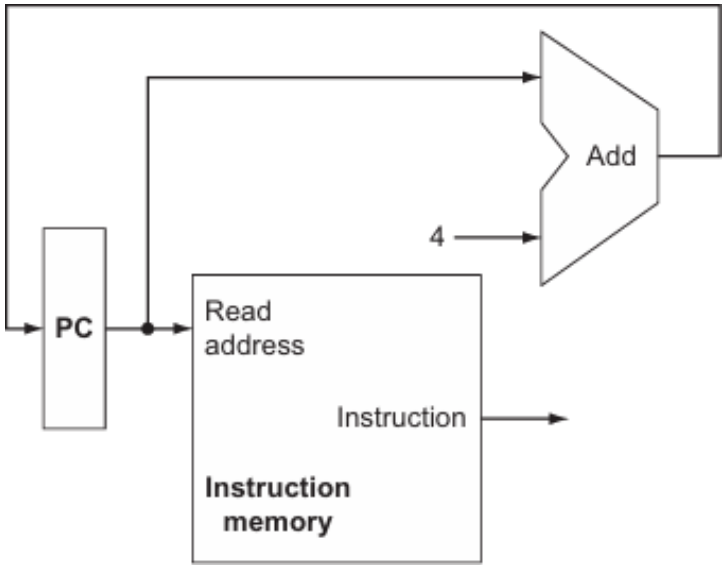
For instance, in Figure 1, two additional units are introduced to implement load and store instructions. These include the data memory unit and the immediate generation unit. The data memory unit acts as a state element with inputs for address and write data, featuring separate read and write controls. Conversely, the immediate generation unit (ImmGen) processes 32-bit instructions, selecting a 12-bit field for load, store, and branch instructions, sign-extending it into a 32-bit output.



a. Data memory unit b. Immediate generation unit

Example:

Combining the PC, Instruction Memory, and Adder in a single unit.



Code:

```
`timescale 1ns / 1ps

module task1 (
input clk, reset,
input [3:0] B,
output [31:0] out
   );

wire [31:0] pc_wire;
wire [31:0] outin;
```

```
PC32 PC(clk, reset, outin, pc_wire);
Adder4 Adder(pc_wire, B, outin);
INST_MEM Mem(pc_wire, reset, out);

endmodule


module Adder4 (
  input [31:0] A,
  input [3:0] B,
  output reg [31:0] Sum
);

  always @(A or B) begin
    Sum <= A + B;
  end

endmodule

module PC32 (
  input clk,
  input reset,
  input [31:0] PC_in,
 output reg [31:0] PC_out
);

  always @(posedge clk or posedge reset) begin
    if (reset) begin
      PC_out <= 32'b0;
    end else begin
      PC_out <= PC_in;
    end
  end

endmodule

module INST_MEM(
  input [31:0] PC,
  input reset,
  output [31:0] Instruction_Code
);
  reg [7:0] Memory [31:0]; // Byte addressable memory with 32 locations
```

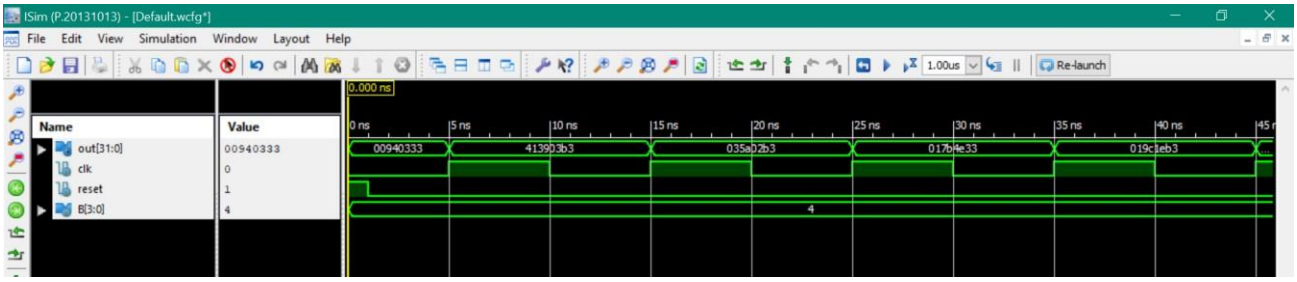
```
assign Instruction_Code =
{Memory[PC+3],Memory[PC+2],Memory[PC+1],Memory[PC]};

   // Initializing memory when reset is one
   always @(reset)
   begin
      if(reset == 1)
      begin
         // Setting 32-bit instruction: add t1, s0,s1
         Memory[3] = 8'h00;
         Memory[2] = 8'h94;
         Memory[1] = 8'h03;
         Memory[0] = 8'h33;
         // Setting 32-bit instruction: sub t2, s2, s3
         Memory[7] = 8'h41;
         Memory[6] = 8'h39;
         Memory[5] = 8'h03;
         Memory[4] = 8'hb3;

         // Setting 32-bit instruction: mul t0, s4, s5
         Memory[11] = 8'h03;
         Memory[10] = 8'h5a;
         Memory[9] = 8'h02;
         Memory[8] = 8'hb3;
         // Setting 32-bit instruction: xor t3, s6, s7
         Memory[15] = 8'h01;
         Memory[14] = 8'h7b;
         Memory[13] = 8'h4e;
         Memory[12] = 8'h33;
         // Setting 32-bit instruction: sll t4, s8, s9
         Memory[19] = 8'h01;
         Memory[18] = 8'h9c;
         Memory[17] = 8'h1e;
         Memory[16] = 8'hb3;
         // Setting 32-bit instruction: srl t5, s10, s11
         Memory[23] = 8'h01;
         Memory[22] = 8'hbd;
         Memory[21] = 8'h5f;
         Memory[20] = 8'h33;
         // Setting 32-bit instruction: and t6, a2, a3
         Memory[27] = 8'h00;
         Memory[26] = 8'hd6;
         Memory[25] = 8'h7f;
         Memory[24] = 8'hb3;
```

```
        // Setting 32-bit instruction: or a7, a4, a5
        Memory[31] = 8'h00;
        Memory[30] = 8'hf7;
        Memory[29] = 8'h68;
        Memory[28] = 8'hb3;
      end
    end

endmodule
```

TestBench:

```
module test123;

// Inputs
reg clk;
reg reset;
reg [3:0] B;

// Outputs
wire [31:0] out;

// Instantiate the Unit Under Test (UUT)
task1 uut (
        .clk(clk),
        .reset(reset),
        .B(B),
        .out(out)
);

always #5 clk = ~clk;

initial begin
        // Initialize Inputs
        clk = 0;
        reset = 1;
        B = 4;

        #1 reset = 0;
        // Wait 100 ns for global reset to finish
        #100 $finish;

        // Add stimulus here
```

```
end

endmodule
```



Immediate Generation:

Immediate generation in RISC-V refers to the process of creating immediate values that are encoded within certain instructions. Immediate values are constant values that are part of an instruction itself, rather than being stored in a register or memory location. These immediate values are typically used for various purposes such as specifying offsets, constants for arithmetic or logical operations, or providing small immediate data directly within an instruction.

In RISC-V, immediate values are often sign-extended and then inserted into specific bit fields within the instruction word. The immediate generation process involves generating the correct immediate value and placing it into the appropriate position within the instruction encoding.

Name (Bit position)	Fields 31:25	24:20	19:15	14:12	11:7	6:0
(a) R-type	funct7	rs2	rs1	funct3	rd	opcode
(b) I-type	immediate[11:0]		rs1	funct3	rd	opcode
(c) S-type	immed[11:5]	rs2	rs1	funct3	immed[4:0]	opcode
(d) SB-type	immed[12,10:5]	rs2	rs1	funct3	immed[4:1,11]	opcode

Code:

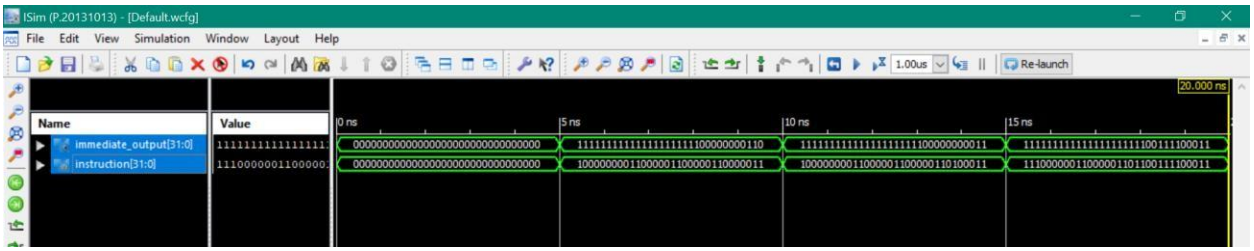
```
`timescale 1ns / 1ps

module imm_gen (
```

```
    input [31:0] instruction,
    output reg [31:0] immediate_output
);

parameter [6:0] LOAD_OPCODE = 7'b0000011;
parameter [6:0] STORE_OPCODE = 7'b0100011;
parameter [6:0] BRANCH_OPCODE = 7'b1100011;

wire [6:0] opcode = instruction[6:0];

wire [11:0] load_im = instruction[31:20];
wire [11:0] store_im = {instruction[31:25],instruction[11:7]};
wire [12:1] branch_im = {instruction[31], instruction[7], instruction[30:25],
instruction[11:8]};

always @(*) begin
    case (opcode)
        LOAD_OPCODE: immediate_output = { {20{load_im[11]}} , load_im};
        STORE_OPCODE: immediate_output = { {20{store_im[11]}} , store_im};
        BRANCH_OPCODE: immediate_output = 2*{{20{branch_im[12]}} ,
branch_im}; // left shift
        default: immediate_output = 32'b0;
    endcase
end

endmodule
```

Test Bench:

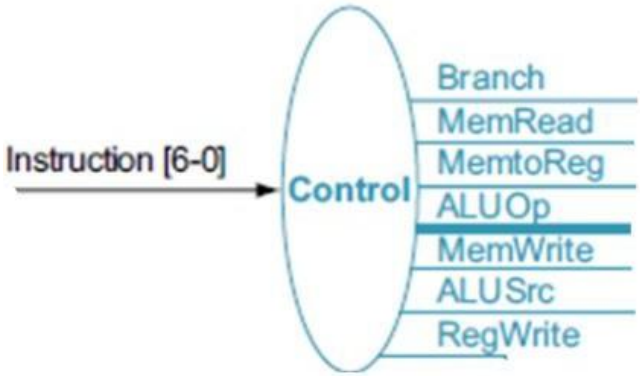
```
`timescale 1ns / 1ps

module imm_gen_test;

        // Inputs
        reg [31:0] instruction;

        // Outputs
        wire [31:0] immediate_output;

        // Instantiate the Unit Under Test (UUT)
        imm_gen uut (
                .instruction (instruction),
                .immediate_output (immediate_output)
        );
```

```
        initial begin
                // Initialize Inputs
                instruction = 0;

                #5 instruction = 32'b10000000011000001100000110000011;
                #5 instruction = 32'b10000000011000001100000110100011;
                #5 instruction = 32'b11100000011000001101100111100011;

                // Wait 100 ns for global reset to finish
                #5 $finish;

                // Add stimulus here

        end

endmodule
```



Control Unit:

The control unit in RISC-V, as in other architectures, is responsible for coordinating and controlling the operation of the CPU. It plays a crucial role in executing instructions by managing the fetch-decode-execute cycle.



Instruction	ALUSrc	Memto-Reg	Reg-Write	Mem-Read	Mem-Write	Branch	ALUOp1	ALUOp0
R-format	0	0	1	0	0	0	1	0
lw	1	1	1	1	0	0	0	0
sw	1	X	0	0	1	0	0	0
beq	0	X	0	0	0	1	0	1

The first row of the table corresponds to the R-format instructions (add, sub, and, and or). For all these instructions, the source register fields are rs1 and rs2, and the destination register field is rd; An R-type instruction writes a register (RegWrite = 1), but neither reads nor writes data memory. When the Branch control signal is 0, the PC is unconditionally replaced with PC + 4; otherwise, the PC is replaced by the branch target if the Zero output of the ALU is also high. The ALUOp field for R-type instructions is set to (10) to indicate that the ALU control should be generated from the funct fields. The second and third rows of this table give the control signal settings for lw and sw. These ALUSrc and ALUOp fields are set to perform the address calculation. MemRead and MemWrite are set to perform memory access. Finally, RegWrite is set for a load to cause the result to be stored in the rd register. The ALUOp field for branch is set for subtract (ALU control = 01), which is used to test for equality. MemtoReg in the last two rows of the table is replaced with X for don't care. This type of don't care must be added by the designer, since it depends on knowledge of how the data path works.

Input or output	Signal name	R-format	lw	sw	beq
Inputs	I[6]	0	0	0	1
	I[5]	1	0	1	1
	I[4]	1	0	0	0
	I[3]	0	0	0	0
	I[2]	0	0	0	0
	I[1]	1	1	1	1
	I[0]	1	1	1	1
Outputs	ALUSrc	0	1	1	0
	MemtoReg	0	1	X	X
	RegWrite	1	1	0	0
	MemRead	0	1	0	0
	MemWrite	0	0	1	0
	Branch	0	0	0	1
	ALUOp1	1	0	0	0
	ALUOp0	0	0	0	1

The control function for the simple single-cycle implementation is completely specified by the above truth table.

ALU Control Unit:

In RISC-V, the ALU (Arithmetic Logic Unit) control unit is responsible for generating control signals that determine the operation to be performed by the ALU based on the instruction being executed. The ALU control unit interprets the opcode and other relevant fields of the instruction to produce the appropriate control signals for the ALU.

Input: The ALU control unit takes as input the opcode field of the instruction, along with any other necessary fields that determine the operation to be performed by the ALU.

Operation Decoding: Based on the opcode and other relevant fields of the instruction, the ALU control unit decodes the operation to be performed by the ALU. This typically involves mapping opcode values to specific ALU operations.

Control Signal Generation: Once the operation is decoded, the ALU control unit generates control signals that specify the ALU operation to be executed. These control signals may include signals to select the ALU operation (e.g., addition, subtraction, etc.), as well as any additional signals needed to configure the ALU for the desired operation.

Output: The control signals generated by the ALU control unit are then used to configure the ALU to perform the specified operation on the input operands.

Lab Tasks:

Task 1: Implement the following Control Unit in Verilog.

Input or output	Signal name	R-format	lw	sw	beq
Inputs	I[6]	0	0	0	1
	I[5]	1	0	1	1
	I[4]	1	0	0	0
	I[3]	0	0	0	0
	I[2]	0	0	0	0
	I[1]	1	1	1	1
	I[0]	1	1	1	1
Outputs	ALUSrc	0	1	1	0
	MemtoReg	0	1	X	X
	RegWrite	1	1	0	0
	MemRead	0	1	0	0
	MemWrite	0	0	1	0
	Branch	0	0	0	1
	ALUOp1	1	0	0	0
	ALUOp0	0	0	0	1

Task 2: Implement the following ALU Control Unit in Verilog.

ALUOp, Funct7, and Funct3 are inputs and Operation is the 4-bit output.

ALUOp		Funct7 field	Funct3 field			
ALUOp1	ALUOp0	I[30]	I[14]	I[13]	I[12]	Operation
0	0	X	X	X	X	0010
X	1	X	X	X	X	0110
1	X	0	0	0	0	0010
1	X	1	0	0	0	0110
1	X	0	1	1	1	0000
1	X	0	1	1	0	0001

Extra Task: Decode the 32-bit instructions from instruction memory into the following outputs .